Version 2.0

January 2026

DATABASE

MODULE 4 - BASIC SQL COMMANDS

Author:

Dr. Ir. Agus Eko Minarno, S. Kom., M. Kom., IPM

Naufal Arkaan

Ismail Dwi Muh. Anugerah

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
INTRODUCTION **3**
OBJECTIVES 3
MODULE TARGETS 3
PREPARATION 3
KEYWORDS 3
MATERIAL **4**
1. WHAT IS SQL? 4
2. DATA DEFINITION LANGUAGE (DDL) 4
3. DATA MANIPULATION LANGUAGE (DML) 6
4. DATA QUERY LANGUAGE (DQL) 6
5. DATA TYPES 7
6. DATE DATA TYPES 8
7. DEFAULT OPTIONS 9
8. CONSTRAINTS 9
9. BASIC SELECT COMMANDS 13
10. INTRODUCTION TO DATA FILTERING (WHERE CLAUSE) 15
11. INTRODUCTION TO DATA SORTING (ORDER BY CLAUSE) 15
SUMMARY MODULE 4 — BASIC SQL **17**
PRACTICE ACTIVITY **18**
A. TUTORIAL ON EXPORTING DATABASES IN ORACLE 11g XE 18
B. ORACLE APEX: PRACTICAL SQL SOLUTIONS (BACKUP) 26
LEARNING RESOURCES AND TUTORIALS **27**
QUERY DOCK **28**
ASSIGNMENT **31**
PRACTICUM GRADING RUBRIC **34**

INTRODUCTION

OBJECTIVES

1. Able to define and manipulate data structures using Data Definition Language (DDL) commands such as `CREATE`, `ALTER`, and `DROP`, as well as Data Manipulation Language (DML) to manage data (`INSERT`, `UPDATE`, `DELETE`).
2. Able to ensure data integrity by applying various constraints to tables, including `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, and `CHECK`.
3. Able to retrieve and display data using basic Data Query Language (DQL) `SELECT` commands, including the use of aliases and arithmetic operators.
4. Able to understand filters and sort retrieved data specifically using the `WHERE` clause (with comparison, logical, and special operators) and the `ORDER BY` clause.

MODULE TARGETS

1. Students can create, modify, and drop tables with the correct data types and constraints.
2. Students can perform complex queries using the `SELECT` command, including basic filtering and sorting, to meet specific data requirements.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. **Oracle 11g XE:**
   LINK_ORACLE 11G XE
   Oracle Apex:
   LINK_ORACLE APEX
   Student Affairs Scheme Link (Table) in the material explanation
   LINK_IMAGE OF THE STUDENT AFFAIRS TABLE
   Link to the Online_Shop schema file for Practice Activity:
   LINK_FILE SCHEMA ONLINE SHOP
3. Strong motivation and commitment to learning

KEYWORDS

Database, Oracle, SQL, DDL, DML, DQL, Constraints, WHERE, ORDER BY.

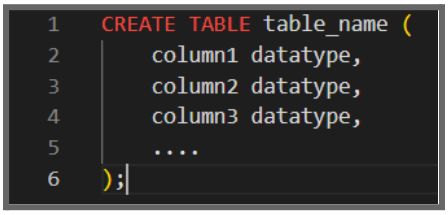
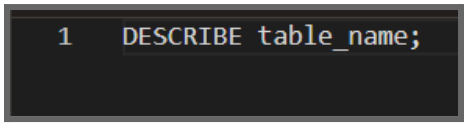
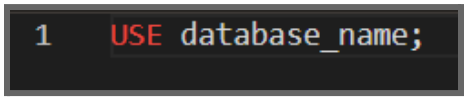
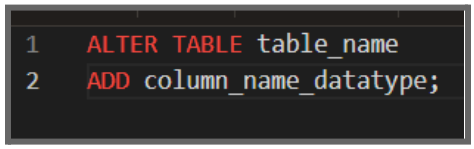
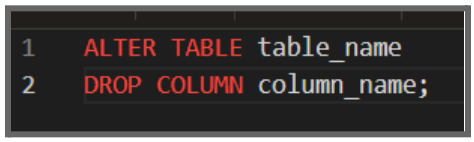
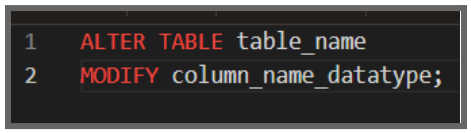
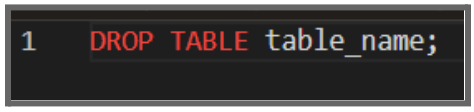
MATERIAL

1. WHAT IS SQL?

SQL stands for **Structured Query Language**. It is the standard language used for managing and manipulating data in **Relational Database Management Systems (RDBMS)**. SQL allows users to perform critical tasks like creating database structures, inserting data, modifying records, and querying information for analysis. SQL commands are categorized into four main sublanguages: DDL, DML, DQL, and DCL.

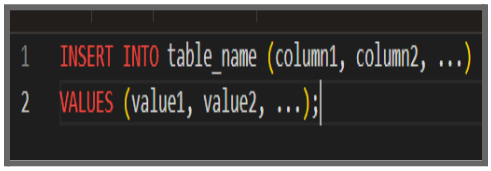
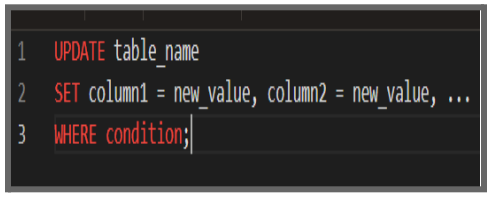
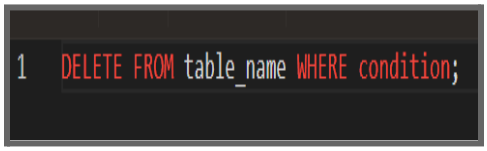
2. DATA DEFINITION LANGUAGE (DDL)

DDL is used to **define, modify, and delete** databases and necessary database objects, such as tables, views, and users. DDL is typically used by database administrators in the creation of database applications. In general, the DDL used is:

Command	Function	Query	Analogy
CREATE	To create a new object in the database, such as a table.		**Preparing a New File Cabinet**: You **purchase an empty file cabinet**, **give it a name** (table name), and **create drawer compartments** (columns) while determining the **type of items** (data type) that can be placed in each drawer.
DESCRIBE	To confirm that the table has been created and view the table details (structure and data types).		**Check the file cabinet labels** to ensure that the drawers (columns) and their sizes (data types) are correct.
USE	To select a specific database/schema as the current active workspace.		**Choose** which filing cabinet you will use and open today.
ALTER	To modify the structure of an existing object (e.g., add, drop, or modify a column/constraint).	**Add column:**  **Delete column:**  **Edit column:** 	**Modify the cabinets** by adding new drawers, removing existing drawers, or changing the size of existing drawers.
DROP	To **permanently delete an object** (structure and all data) from the database.		**Discard the entire filing cabinet** and all the files in it (Total deletion).

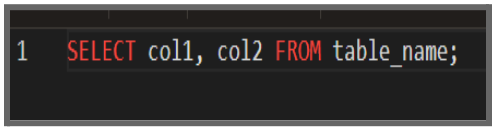
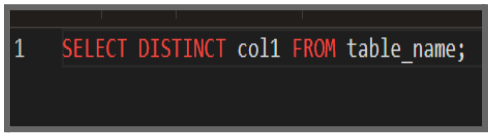
3. DATA MANIPULATION LANGUAGE (DML)

DML is used to **manipulate data** stored in a table, not the table structure itself. These commands are essential for routine data management. Common commands are as follows:

Command	Function	Query	Analogy
INSERT	Used to **add new rows of data** into a table.		You are **placing a new file (a new record/row)** into the designated drawers (columns) of the filing cabinet.
UPDATE	Used to **modify existing data** within the rows of a table. Requires the WHERE clause to specify which rows to change.		You are **opening a file and changing the content** inside it, but only the specific file that meets the condition (e.g., changing the address on the 'Steven' file).
DELETE	Used to **remove existing rows of data** from a table. Requires the WHERE clause to specify which rows to remove.		You are **removing a specific file** from a drawer, but the drawer (table structure) itself remains.

4. DATA QUERY LANGUAGE (DQL)

DQL is specifically used to **retrieve or query** data from the database. The primary and most common command is SELECT.

Command	Function	Query	Analogy
`SELECT*`	To **retrieve all columns** and all rows from the specified table.		**Get All the Information:** "Please **get all the files** and **provide all the contents** (all columns) without exception."
`SELECT (Columns)`	To retrieve data from **specific columns** only.		**Get Specific Information**: "Please get all the files, but I only need the **Name Column and Salary Column**."
`SELECT DISTINCT`	To retrieve unique (non-duplicate) values from the specified column(s).		**List Without Repetition**: "Please list all **job titles** in this company, but **do not repeat** any (only list the different ones)."

5. DATA TYPES

Data Types specify the kind of data that can be stored in a column, such as numbers, characters, or dates. Choosing the correct data type is crucial for efficient storage and accurate data manipulation.

Data Type	Description	Analogy
`VARCHAR2 (size)`	Variable-length character data. Stores text and characters up to a specified maximum size (e.g., 255). It saves space because it only uses the size required by the actual data.	**Flexible Plastic Bag**: A drawer that can **adjust its size to fit the contents** (text). If you write 'A' (1 character), only 1 space is used, and the rest is saved.
`CHAR (size)`	Fixed-length character data. Always reserves the maximum specified size, even if the actual data is shorter.	**Rigid Paper Box**: Drawers that are **always the same size** (for example, 50 spaces). Even if you only write 'A', it still takes up 50 spaces.

`NUMBER (p, s)`	Numeric data used for storing whole numbers or decimals. *p* is precision (total digits), and *s* is scale (digits to the right of the decimal point).	**Special Cash Drawer**: A drawer that only accepts **numbers** (all types of currencies). We determine how many total digits can be entered (p) and how many digits after the decimal point (s).
DATE	Stores date and time values, including year, month, day, hour, minute, and second.	**Time Stamp**: Used to record the exact **date and time** of an event (for example, when a file was created or modified).

6. DATE DATA TYPES

These types provide more advanced time storage capabilities, often including timezone information and fractional seconds.

Data Type	Description	Analogy
TIMESTAMP	Stores date and time data with **seconds and fractional seconds** (very precise time measurement).	**Stopwatch Watch**: More accurate than a regular time stamp, because it records time down to fractions of a second **(milliseconds)**.
INTERVAL YEAR TO MONTH	Stores a span or duration of time in terms of **years and months** (e.g., "3 years and 2 months").	**Project Duration**: Stores the **time period** between two dates measured only in Years and Months.
INTERVAL DAY TO SECOND	Stores a span or duration of time in terms of **days, hours, minutes, and seconds**.	**Travel Duration**: Stores the **time period** between two events measured in Days, Hours, Minutes, and Seconds.

7. DEFAULT OPTIONS

DEFAULT option is used when creating a table to **automatically assign a value** to a column if a new row is inserted and **no specific value** is provided for that column. Example:

```
CREATE TABLE Students (
    StudentID NUMBER PRIMARY KEY,
    Name VARCHAR2(100) NOT NULL,
    EnrollmentDate DATE DEFAULT SYSDATE,
    StudentStatus VARCHAR2(10) DEFAULT 'Active' );
```

Analogy:

Automatic Status Setting: When a student is first inserted into the system, their **StudentStatus** is **automatically set to 'Active'** unless specified otherwise. The **EnrollmentDate** is automatically set to the **current date**.

8. CONSTRAINTS

Constraints in SQL are rules or restrictions applied to tables to ensure data remains **consistent and accurate**. They ensure that values in certain columns comply with specific rules, such as not being NULL, being unique, or referring to values in other tables.

Constraints	Function	Query	Analogy
`PRIMARY KEY`	Uniquely identifies each row in a table. It cannot be NULL and must be unique.	`id NUMBER PRIMARY KEY`	**Student Identification Number (NIS)**: The main identity card that **must be owned** and **cannot be duplicated**.
`FOREIGN KEY`	Creates a link (relationship) between two tables. It must refer to a value that already exists in the primary key column of another table.	`FOREIGN KEY (id_major) REFERENCES Major(id)`	**Graduate Certification**: A column that **only accepts values** (e.g., Department ID) that are **already recorded as valid** in the reference table (Department Table).
`NOT NULL`	Ensures that the column **cannot contain a null (empty) value**.	`name VARCHAR2(50) NOT NULL`	**Required Fields**: Sections of the form that must be filled in and **cannot be left blank**.
`UNIQUE`	Ensures that all values in a column are **unique** (no duplicates allowed), but it **can** contain NULL values.	`email VARCHAR2(100) UNIQUE`	**Mobile Number**: Everyone must have a different mobile number, but **not everyone is required** to include it (optional).
`CHECK`	Defines a condition that must be met by the data inserted into the column.	`CHECK (age >= 17)`	**Age Restriction Filter**: A rule that ensures the data entered meets logical requirements (for example, the Age value must be above 17 years old).

Implementing Constraints: Step-by-Step Guide:

This guide details the creation of the core academic tables, defining all necessary constraints (`PK`, `FK`, `NOT NULL`, `CHECK`, `UNIQUE`), and testing data integrity.

Step 1: Establish Base Tables (MAJORS and LECTURERS)

Create the foundational tables first, as they contain the primary keys referenced by all other tables.

```sql
CREATE TABLE MAJORS (
    MajorID NUMBER(3) PRIMARY KEY,
    MajorName VARCHAR2(100) NOT NULL UNIQUE
);
```

```sql
CREATE TABLE LECTURERS (
    LecturerID NUMBER(5) PRIMARY KEY,
    LecturerName VARCHAR2(100) NOT NULL,
    Email VARCHAR2(100) UNIQUE,
    MajorID NUMBER(3),
    -- Foreign Key Definition
    CONSTRAINT FK_LecMajor FOREIGN KEY (MajorID) REFERENCES MAJORS (MajorID)
);
```

Step 2: Create Transactional and Entity Tables (STUDENTS and COURSES)

Implement tables that rely on the base tables, including more complex constraints like CHECK and DEFAULT.

```sql
CREATE TABLE STUDENTS (
    NIM VARCHAR2(10) PRIMARY KEY,
    FullName VARCHAR2(100) NOT NULL,
    MajorID NUMBER(3),
    GPA NUMBER(3, 2) CHECK (GPA BETWEEN 0.00 AND 4.00),
    EnrollmentYear NUMBER(4),
    -- Default Option
    Status VARCHAR2(10) DEFAULT 'Active',
    -- Foreign Key Definition
    CONSTRAINT FK_StudMajor FOREIGN KEY (MajorID) REFERENCES MAJORS (MajorID)
);
```

```sql
CREATE TABLE COURSES (
    CourseCode VARCHAR2(5) PRIMARY KEY,
    CourseName VARCHAR2(100) NOT NULL,
    Credits NUMBER(1) CHECK (Credits BETWEEN 2 AND 4),
    LecturerID NUMBER(5),
    -- Foreign Key Definition
    CONSTRAINT FK_CourseLec FOREIGN KEY (LecturerID) REFERENCES LECTURERS (LecturerID)
);
```

Step 3: Demonstrate Data Integrity Violation

After table creation, test the system's ability to reject bad data based on the defined rules.

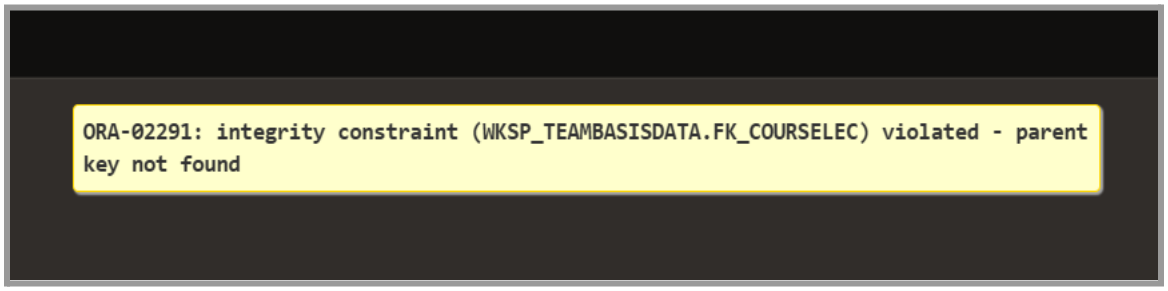
A. Foreign Key Violation (Referential Integrity)

Goal: Attempt to enroll in a course with a non-existent lecturer ID.

- **Assumption:** Lecturer ID 999 does not exist in the **LECTURERS** table.
- **Violating Query:**

```
1   INSERT INTO COURSES (CourseCode, CourseName, Credits, LecturerID)
2   VALUES ('DB101', 'Database Fundamentals', 3, 999);
```

- **Expected Outcome:** The database will reject the insertion.



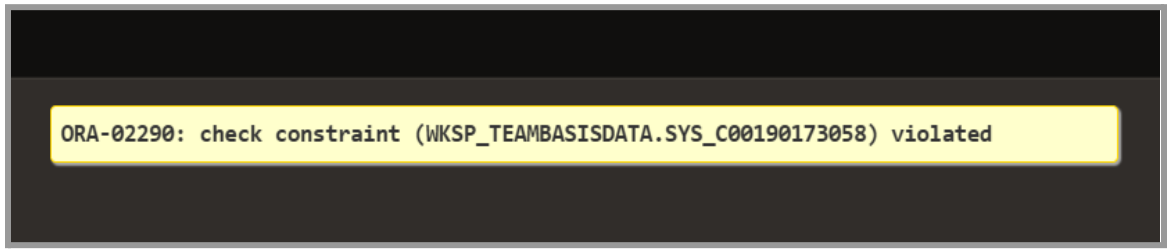
B. CHECK Constraint Violation (Domain Integrity)

Goal: Attempt to `insert` a student with an invalid GPA value (e.g., above 4.00).

- **Violating Query:**

```
1   INSERT INTO STUDENTS (NIM, FullName, MajorID, GPA)
2   VALUES ('2025101', 'Andi Nugraha', 101, 4.50);
```

- **Expected Outcome:** The database will reject the insertion.



9. BASIC SELECT COMMANDS

1. Retrieving All Columns (SELECT *)

Displays all columns and all rows from the specified table.

- Query:

```
1    SELECT * FROM MAJORS;
```

- Expected Output:

MAJORID	MAJORNAME
101	Informatika
102	Sistem Informasi
103	Teknik Elektro

- **Purpose:** To retrieve all details of every Major.

2. Retrieving Specific Columns

Retrieves only the necessary column names separated by commas.

- **Query:**

```
1    SELECT LecturerName, Email
2    FROM LECTURERS;
```

- **Expected Output:**

LECTURERNAME	EMAIL
Prof. Cahyo Wibowo	cahyo.w@univ.ac.id
Dr. Budi Santoso	budi.s@univ.ac.id
Ir. Siti Aisyah	siti.a@univ.ac.id

- **Purpose:** To retrieve only the name and email of the lecturers.

3. Eliminating Duplicates (SELECT DISTINCT)

Eliminates duplicate rows in the result set, ensuring each unique value appears only once.

- **Query:**

```
1   SELECT DISTINCT MajorID
2   FROM LECTURERS;
```

- **Expected Output:**

MAJORID
101
102

- **Analogy:** "Show all the different Major IDs that lecturers belong to, but do not repeat the same ID."

4. Arithmetic Operations

Performs arithmetic calculations (**+, -, *, /**) on number columns.

- **Query:**

```
1   SELECT CourseName, Credits, Credits * 16 AS "Total_Hours"
2   FROM COURSES;
```

- **Expected Output:**

COURSENAME	CREDITS	Total_Hours
Basis Data	3	48
Analisis Sistem	4	64
Jaringan Komputer	2	32
Pemrograman Dasar	3	48

- **Analogy:** "Take the value from the **Credits** column and **multiply it by 16** to create a new output column named 'Total_Hours'."

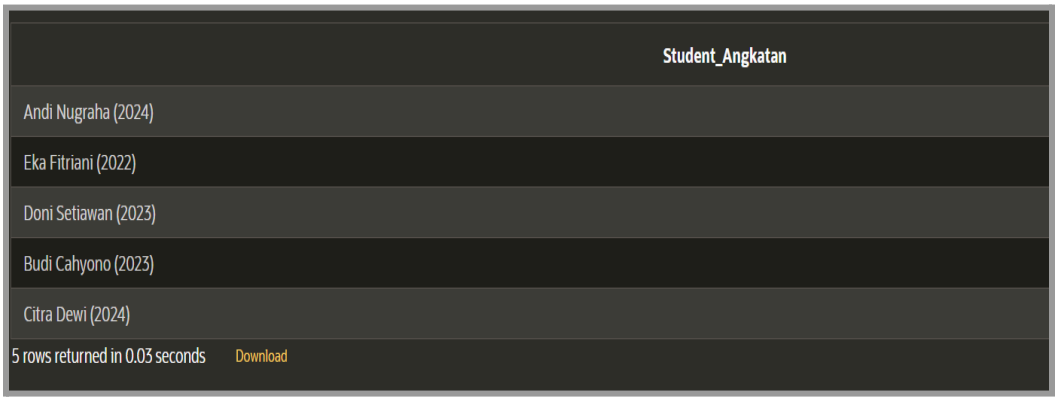
5. Concatenation (||)

Combines two or more columns into a single output string.

- **Query:**

```
SELECT FullName || ' (' || EnrollmentYear || ')' AS "Student_Angkatan"
FROM STUDENTS;
```

- **Expected Output:**



Student_Angkatan
Andi Nugraha (2024)
Eka Fitriani (2022)
Doni Setiawan (2023)
Budi Cahyono (2023)
Citra Dewi (2024)

5 rows returned in 0.03 seconds Download

- **Analogy:** "Stitch the Student Name, a parenthesis, and the Enrollment Year together to form one detailed string."

10. INTRODUCTION TO DATA FILTERING (WHERE CLAUSE)

The WHERE clause is an essential component of the DQL (SELECT) command. It acts as a **filter** in your SQL query, specifying which rows must meet a certain **condition** to be included in the final result set.

Operator	Function	Query	Analogy
WHERE	Selecting rows that meet a certain condition.	`SELECT * FROM STUDENTS WHERE GPA > 3.50;`	Take all the files, **BUT** only students with a GPA above 3.50.
AND / OR	Combining two or more conditions.	`WHERE MAJOR = 'Informatics' AND ENROLLMENT_YEAR = 2024;`	Take the file if the major is “Informatics” **AND** the year of entry is 2024.

11. INTRODUCTION TO DATA SORTING (ORDER BY CLAUSE)

The ORDER BY clause is used to define the **order** in which the result set is displayed. This operation is performed **after** the data has been filtered and retrieved, making it the final logical step in presenting data.

Keyword	Function	Query	Analogy
ORDER BY	Sort results based on the specified column.	SELECT NAME, GPA FROM STUDENTS ORDER BY GPA DESC;	After the data is collected, **SORT** it based on GPA.
DESC / ASC	DESC (Descending): from largest/newest to smallest/oldest. ASC (Ascending): from smallest/oldest to largest/newest (default).	... ORDER BY ENROLLMENT_YEAR DESC;	Sort students by Year of Entry FROM **NEWEST** to OLDEST.

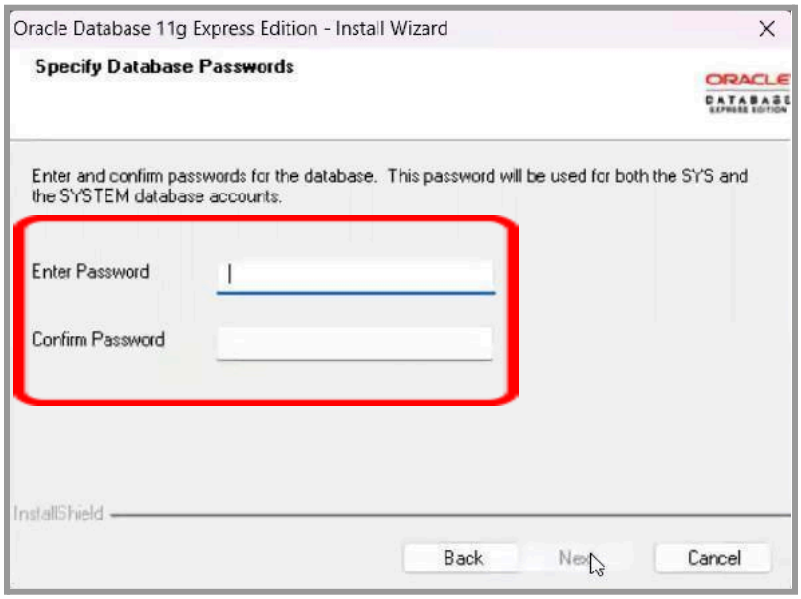
SUMMARY MODULE 4 — BASIC SQL

Query	How to Read in Indonesian (Brief Analogy)
`CREATE TABLE Students (...);`	Buat tabel Students
`ALTER TABLE Students ADD Email...;`	Tambah kolom Email di tabel Students
`DROP TABLE Students;`	Hapus tabel Students
`INSERT INTO Students VALUES (...);`	Tambahkan data baru ke tabel Students
`UPDATE Students SET ... WHERE NIM='2024...';`	Ubah data mahasiswa tertentu
`DELETE FROM Students WHERE NIM='2024...';`	Hapus satu data mahasiswa
`SELECT * FROM Students;`	Tampilkan semua data Students
`SELECT FullName, GPA FROM Students;`	Tampilkan hanya Nama dan GPA
`SELECT DISTINCT MajorID FROM Students;`	Tampilkan daftar jurusan tanpa duplikat
`SELECT * FROM Students WHERE GPA > 3.5;`	Tampilkan mahasiswa dengan IPK di atas 3.5
`WHERE GPA > 3.5 AND MajorID = 101`	Filter: IPK > 3.5 dan jurusan 101
`ORDER BY GPA DESC`	Urutkan data dari IPK tertinggi ke terendah
`PRIMARY KEY (NIM)`	NIM sebagai identitas unik mahasiswa
`FOREIGN KEY (MajorID)`	Hubungkan ke tabel Jurusan
`NOT NULL`	Kolom wajib diisi
`UNIQUE`	Nilai tidak boleh kembar
`CHECK (GPA BETWEEN 0 AND 4)`	Nilai hanya boleh antara 0 sampai 4

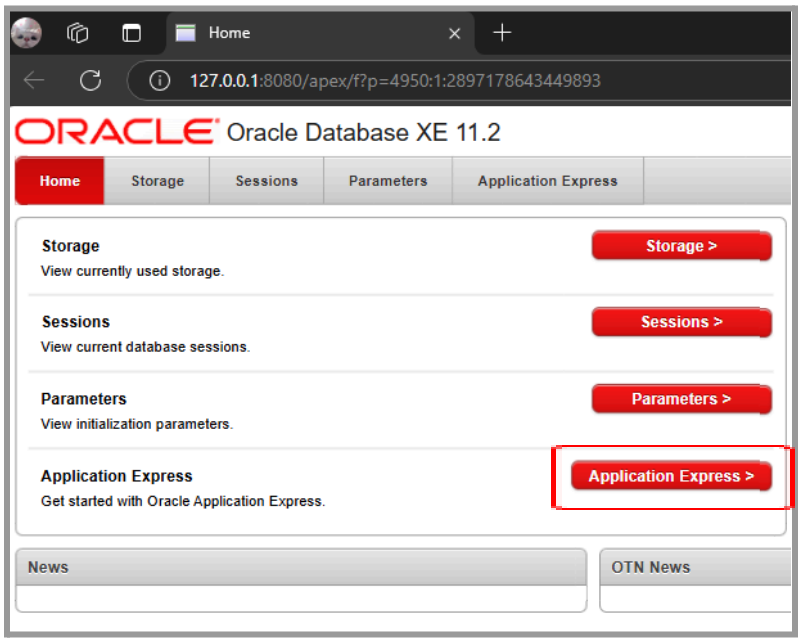
PRACTICE ACTIVITY

A. TUTORIAL ON EXPORTING DATABASES IN ORACLE 11g XE

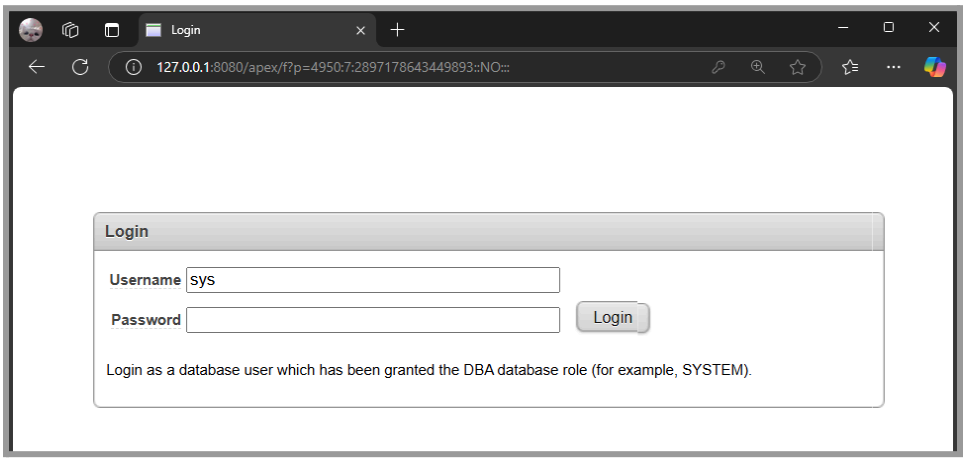
1. ***Download Oracle 11g XE*** *and then install it (* LINK_ORACLE 11G XE *). When the installation process appears as shown below, enter the password as recommended, which is* ***SYS*** *or* ***SYSTEM*** *(capital letters are optional). This password will be used to log in to the Oracle database,* ***so it is important to remember it.***



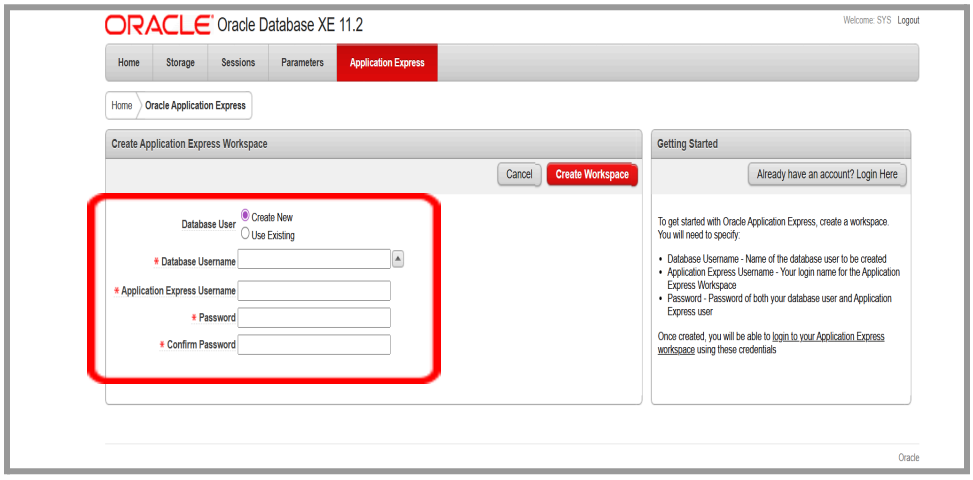
2. **Run Oracle 11g XE and select "Application Express" on the main page.**



3. **Log in using the account created in step 1.**

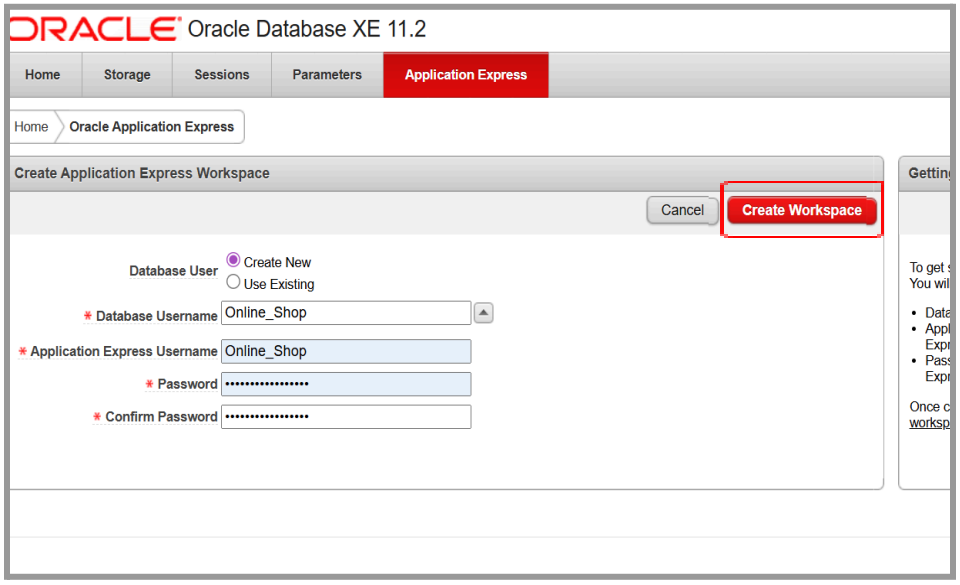


4. **Create an empty database as a workspace.**

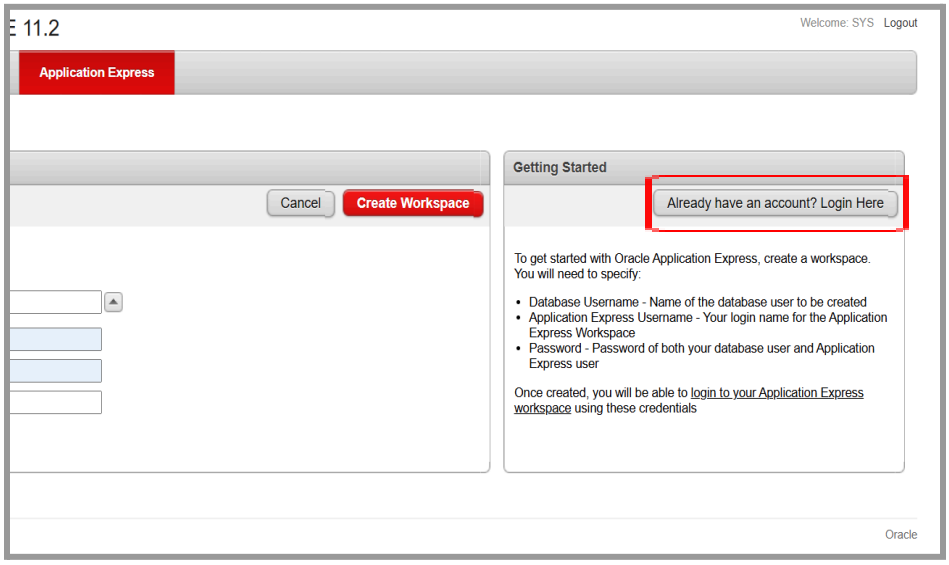


- **Database User -** There are options to Create New (create a new database) or Use (use an existing database such as HR). In this exercise, we will create a new database.
- **Database Username -** You can use the provided database or create your own username.
- **Application Express Username -** To name the Workspace, try to make this name the same as the database username.
- **Procedures for Creating a New Workspace:**
    1. Select the “Create New” radio button under “Database User”.
    2. Enter the user name for the new database schema to be created (Example: **Online_Shop**).
    3. Enter the login name you will use to log in to the APEX Workspace environment (Example: **Online_Shop**).
    4. Enter the passwords for both users above (Database User and Application Express User). **Use the same password you**

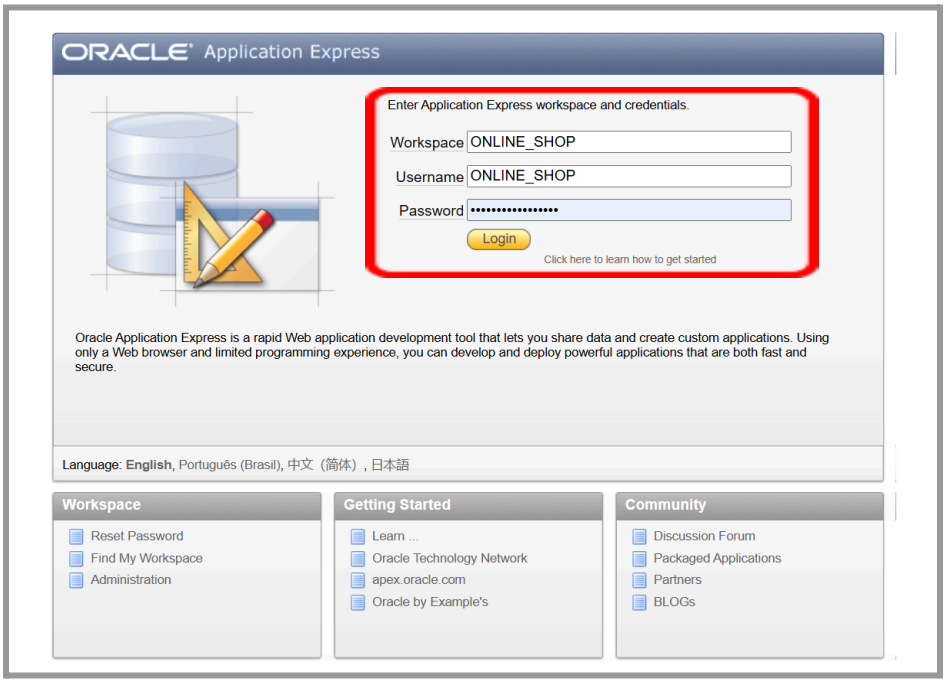
use to log in to the Oracle database, and don't forget the password.



5. After all required fields (Database Username, Application Express Username, and Password) are filled in, click the **"Create Workspace"** button (red button).
6. After the process is complete, click the **"Already have an account? Login Here"** button.



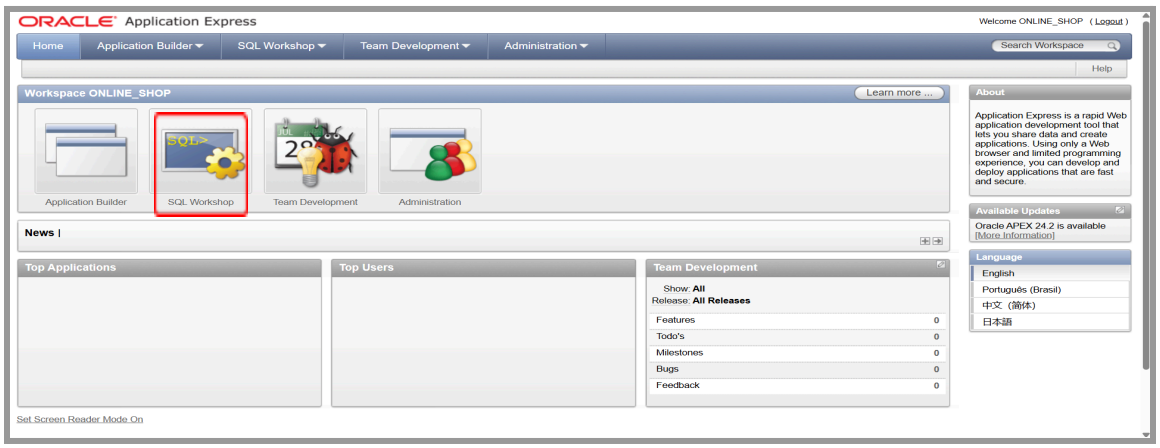
7. Enter Credentials:



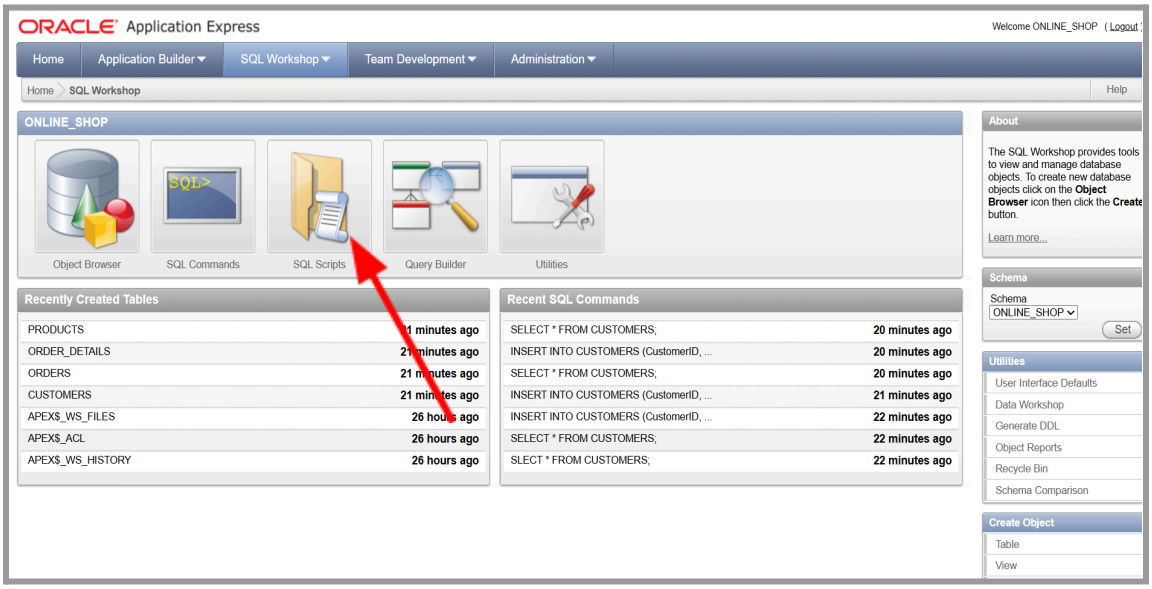
- Workspace: Enter the name of the workspace you just created (usually the same as your Application Express Username, Example: **ONLINE_SHOP**).
- Username: Enter your Application Express Username (Example: **ONLINE_SHOP**).
- Password: Enter the password you created.
- Click **login** to enter the workspace page.

5. How to Export Databases to SQL Scripts

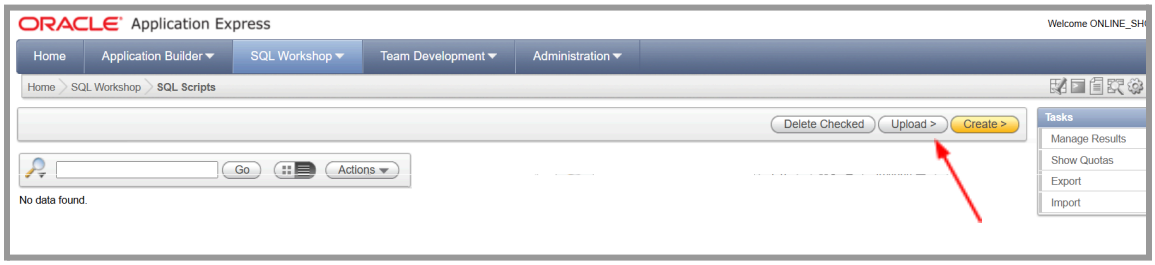
1. On the Workspace (ONLINE_SHOP) main page, click the **"SQL Workshop"** icon or menu.



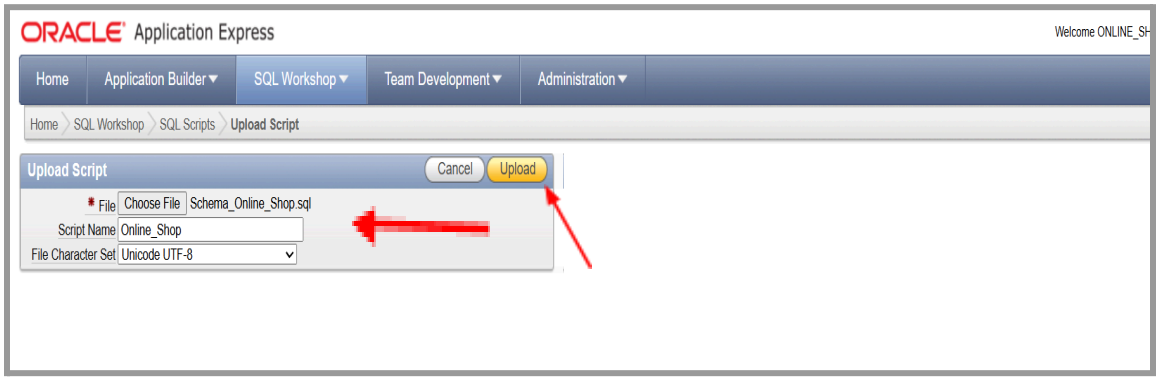
2. In the SQL Workshop menu, click on the **“SQL Scripts”** submenu.



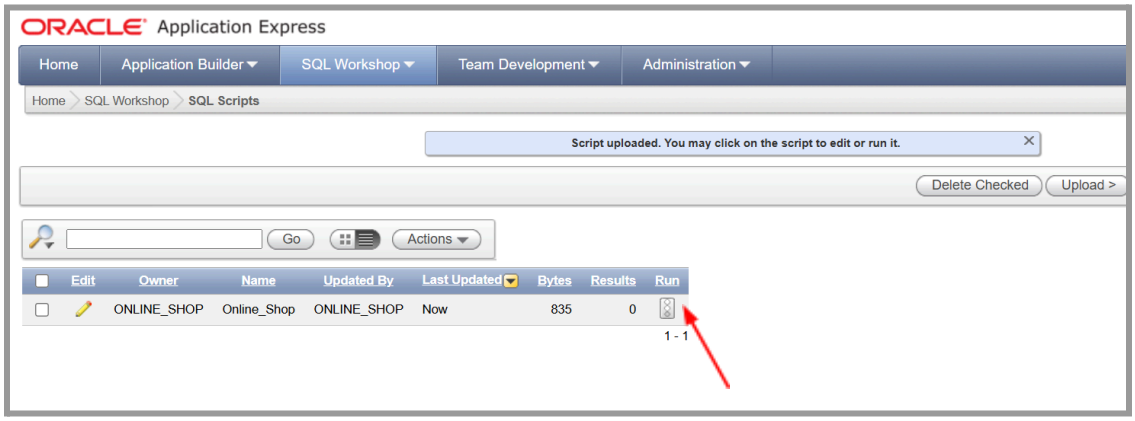
3. Click the **“Upload”** button.



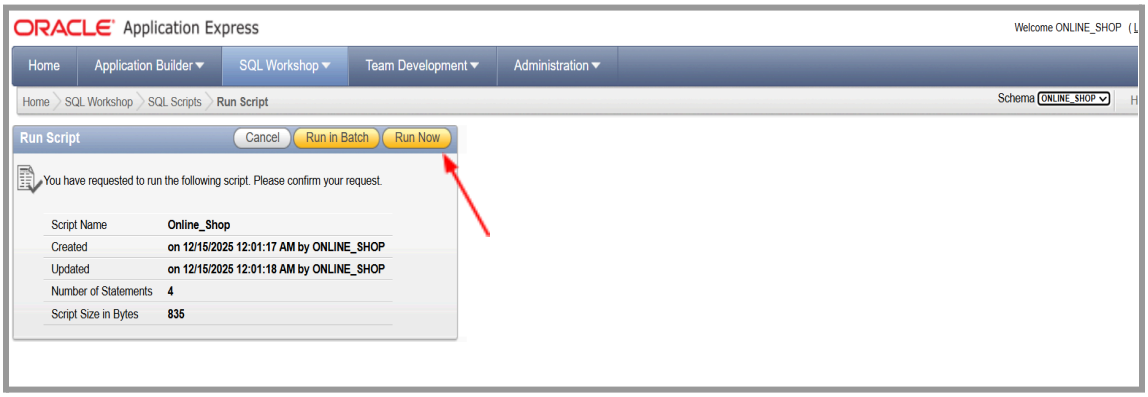
4. Download and then upload this schema file (LINK_FILE_SCHEMA ONLINE SHOP). Give the script the name **“Online_Shop”**. After that, click the “Upload” button.



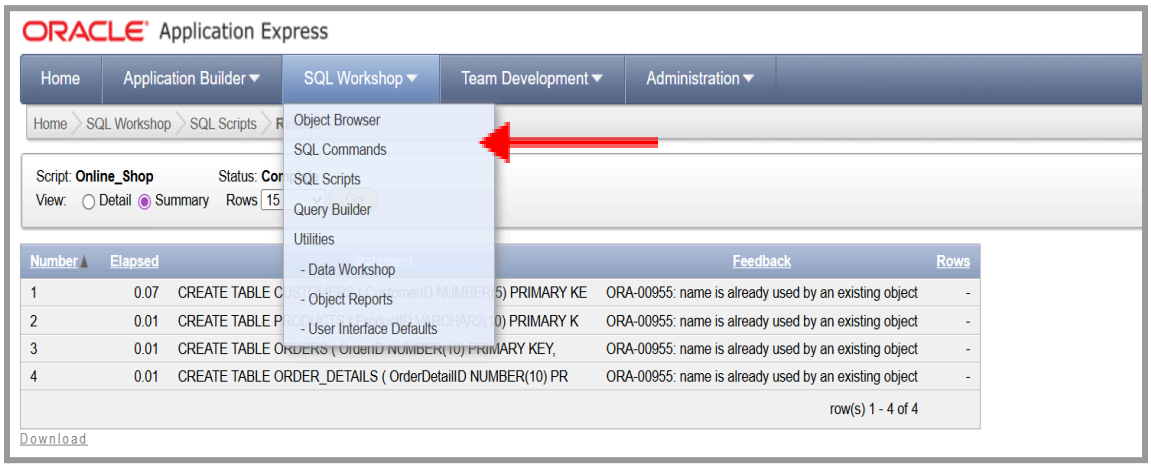
5. Click **“RUN”**



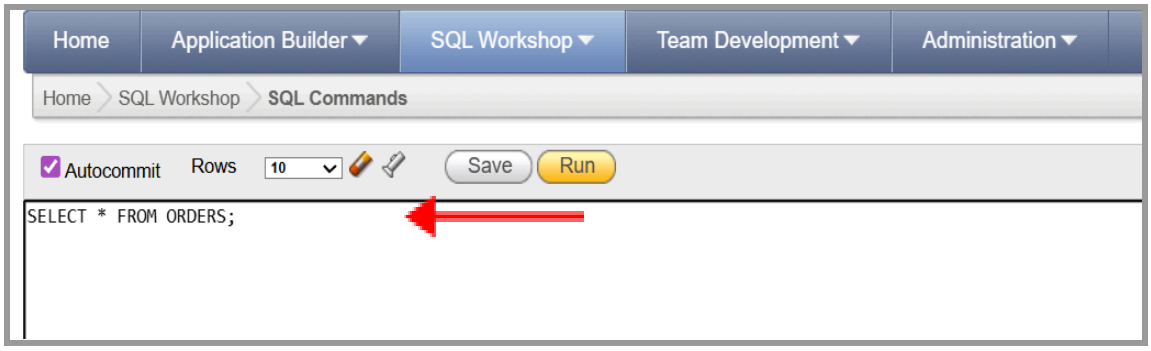
6. Click button **“Run Now”**



7. Now that the Online_Shop schema has been successfully exported, the next step is to click **“SQL Commands”** to write the query code.

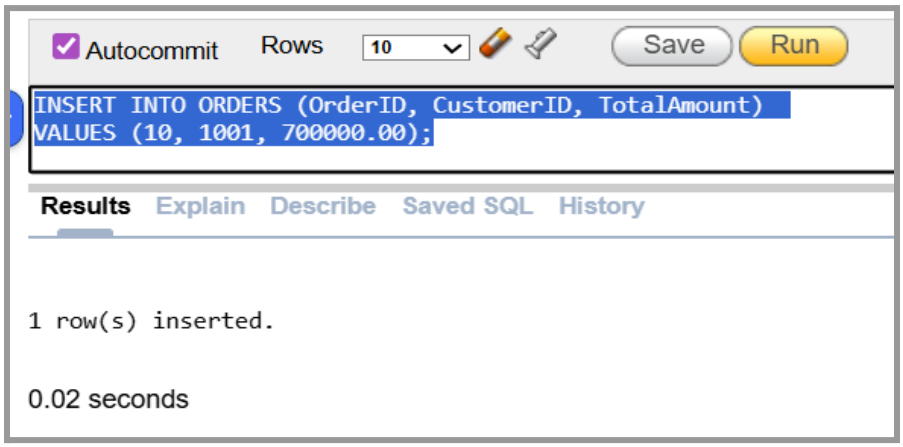


8. Check one of the tables (entities) to ensure that the database export was created using the query.

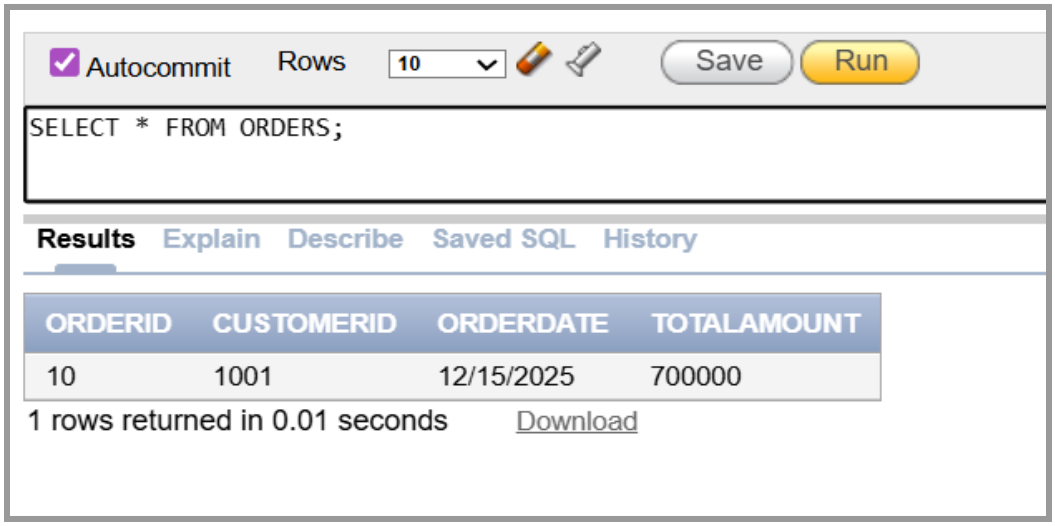


9. When you run the query for the first time, the result you will get is **"data not found"**, which means that you have not entered any data into the database because the database you created is still empty.

10. Try entering one piece of data into one of the entity databases to see the form of the database you have built. **You can select the query syntax to be executed and then click the "Run" button**.

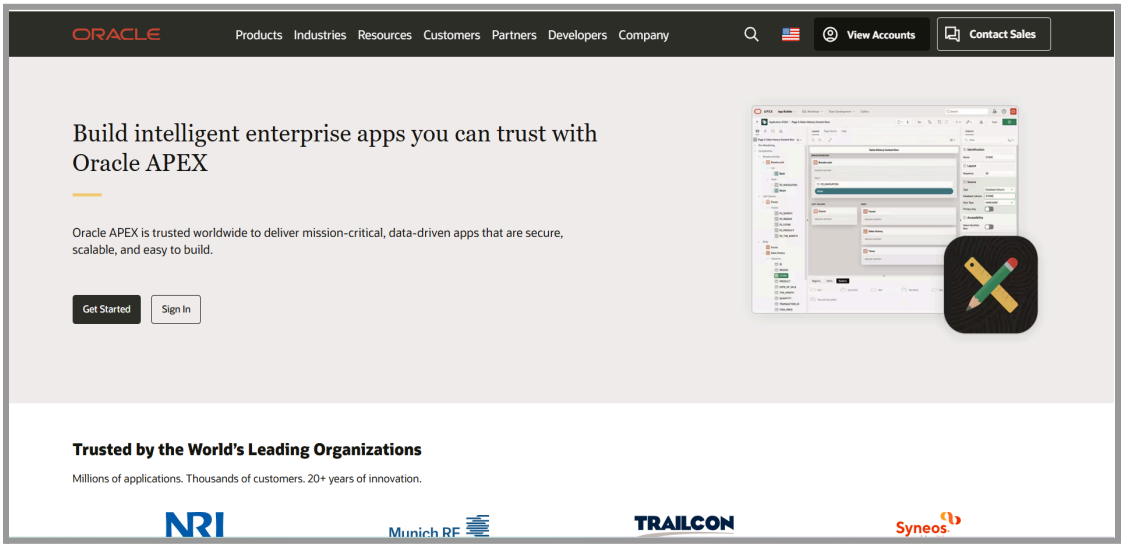


11. If there are no red error messages, then you have successfully exported the database. Validate by calling the orders table.



B. ORACLE APEX: PRACTICAL SQL SOLUTIONS (BACKUP)

Oracle Application Express (APEX) is a web-based interface that allows you to run SQL commands directly through your browser. LINK_ORACLE APEX



Key Points:

1. **Cross-Platform Solution:** APEX is the perfect solution for macOS and other operating system users who have difficulty installing Oracle Database XE locally. All you need is an internet connection.
2. **Backup Environment:** APEX serves as a stable backup practice environment. If your local Oracle 11g XE database encounters an error, APEX is ready to use.
3. **Syntax Similarity:** The SQL commands you use in APEX (DDL, DML, DQL) are exactly the same as those used in Oracle 11g XE. The only difference is in the tool interface.
4. **Set Up a Backup:** Don't forget to create your free APEX Workspace account as a backup in case of problems with your local Oracle XE installation.

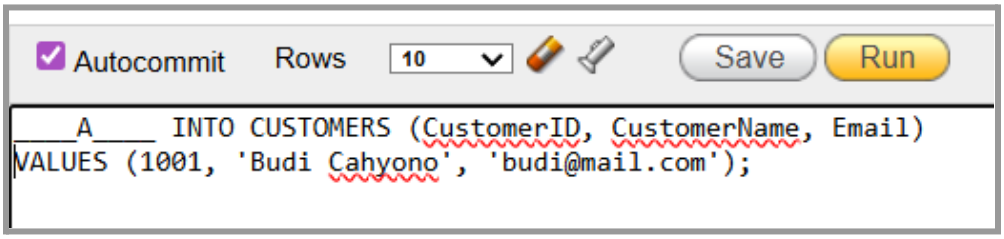
LEARNING RESOURCES AND TUTORIALS

LINK_W3SCHOOLS_SQL **(Recommendations)**
LINK_ORACLE_LIVE SQL **(Recommendations)**
LINK_SQLZoo **(Recommendations)**
LINK_THOUGHTSPOT_SQL TUTORIAL
LINK_GEEKSFORGEEKS_SQL

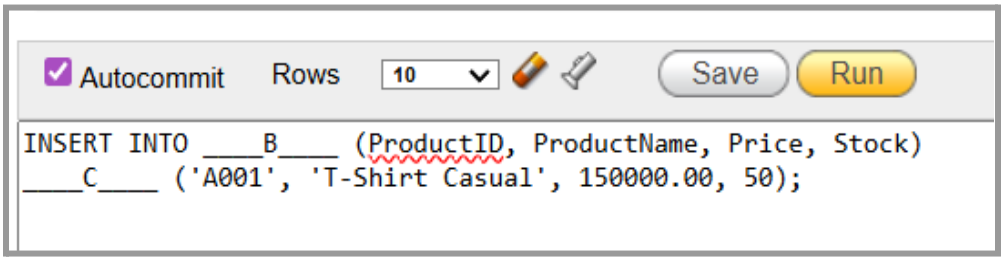
QUERY DOCK

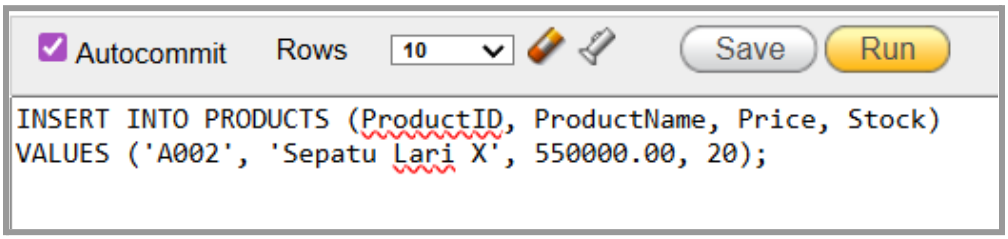
In the **Practice Activity** section, we have entered data into the **ORDERS** table. Now, your task is to complete the data in the other three tables so that our Online Shop database is complete. Fill in the blanks (____) with the appropriate keywords or values!

1. **Insert Customer Data (CUSTOMERS)**

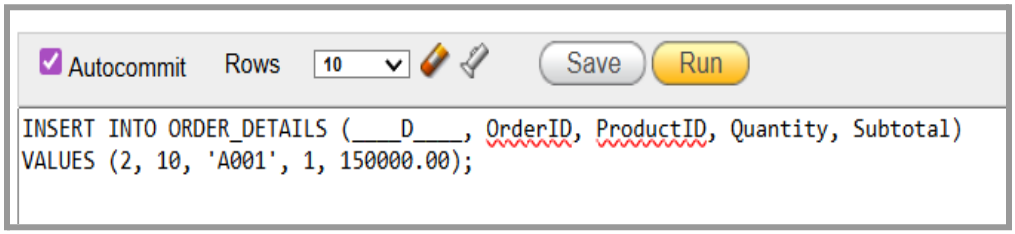


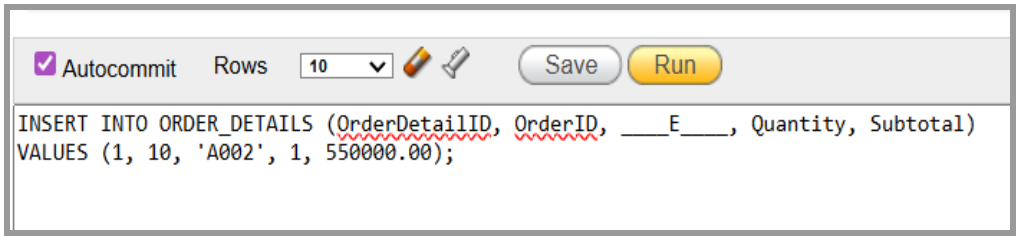
2. **Insert Product Data (PRODUCTS)**





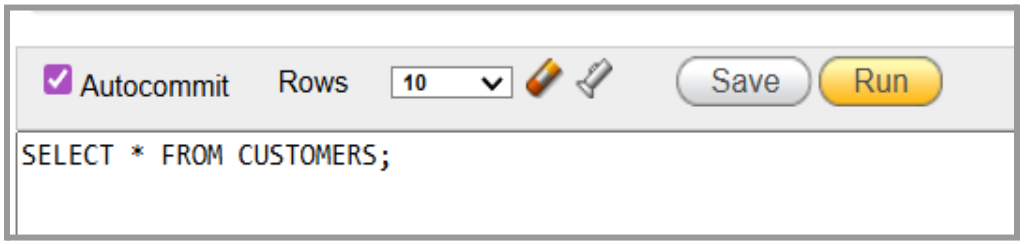
3. **Insert Order Details Data (ORDER_DETAILS)**





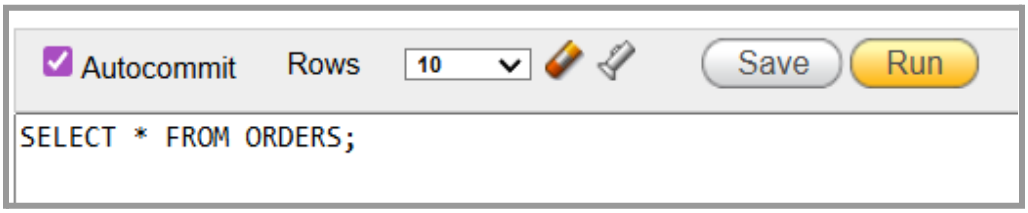
After all data has been successfully entered, run the following commands in sequence to display the entire contents of your table. Make sure all data (including the blank data you just filled in) appears correctly!

1. **Show all Customers:**

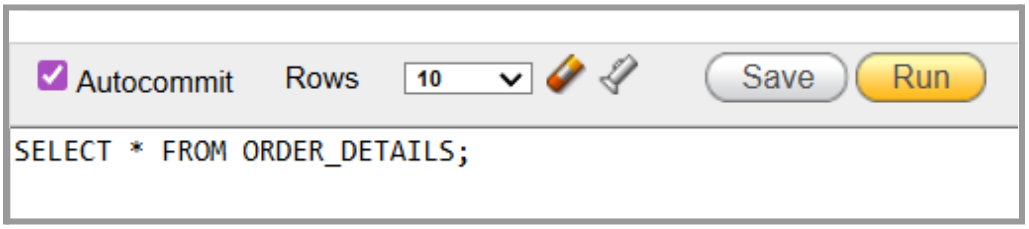


2. **Show all Products:**

3. **Show all Orders:**



4. **Show all Order Details:**



Note: If all the data appears, congratulations! You have succeeded. Show the results to the Assistant.

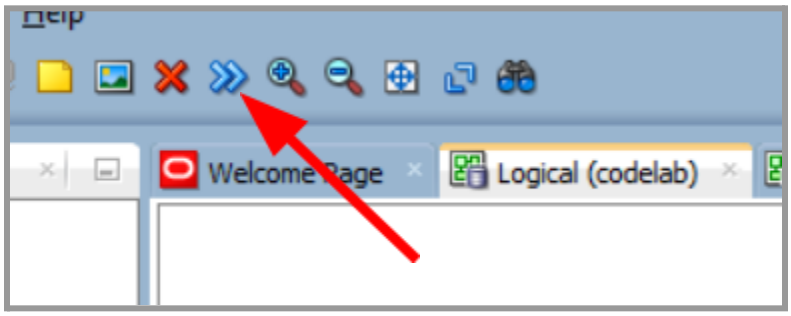
ASSIGNMENT

Task instructions:

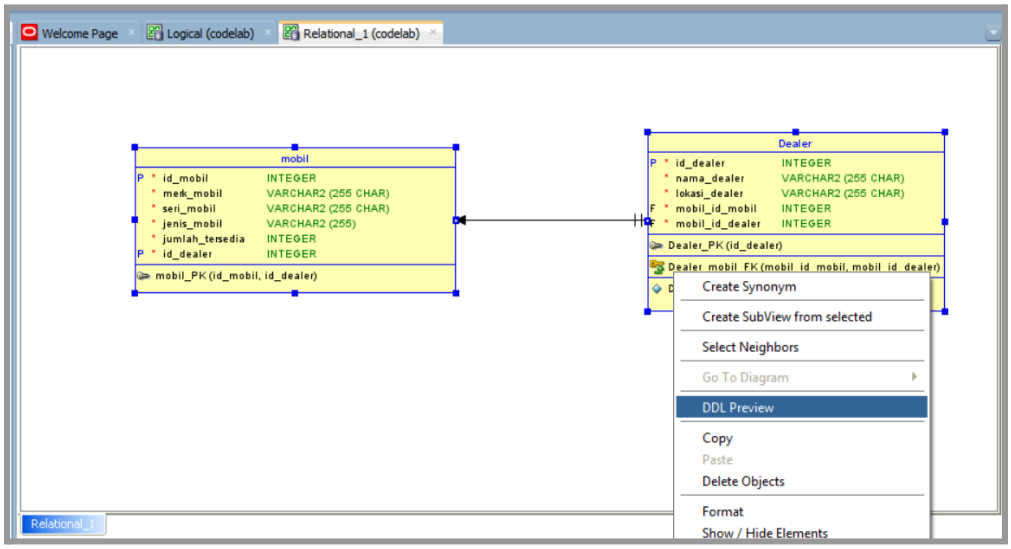
1. Add at least 3 data to each main table (`INSERT`).
2. Display all data from one of the tables (`SELECT`).
3. Display data with certain conditions (`WHERE`).
4. Change one data (`UPDATE`).
5. Delete one data (`DELETE`).

This task is based on the schema theme you created in the previous module (in SQL Developer). Please upload your schema DDL file to Oracle 11g XE. To upload the DDL file, follow these steps (the image below is only an example, please use the schema theme you have created):

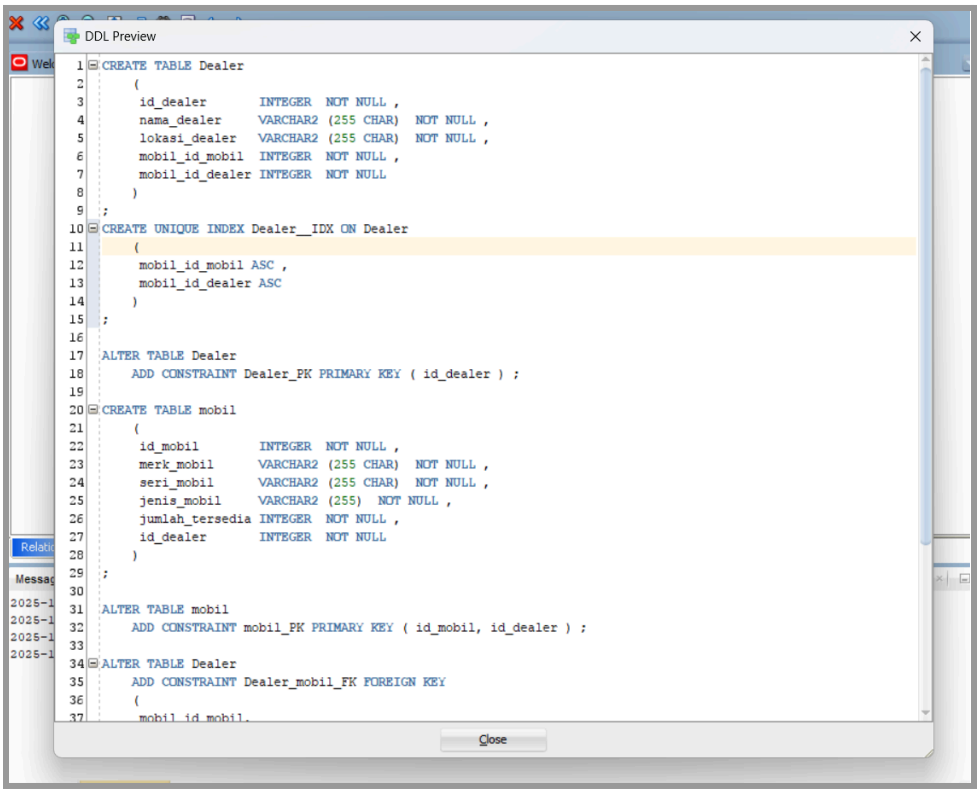
1. **Perform the CDM to PDM conversion as taught in the module 2 tutorial.**



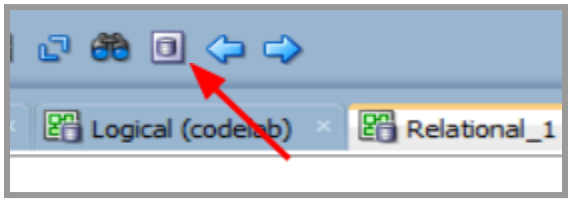
2. **Then, switch to the Relational Data Model, select all the tables/entities you have created by drafting or pressing CTRL + A, right-click and select the "DDL Preview" menu. Copy all available DDLs.**



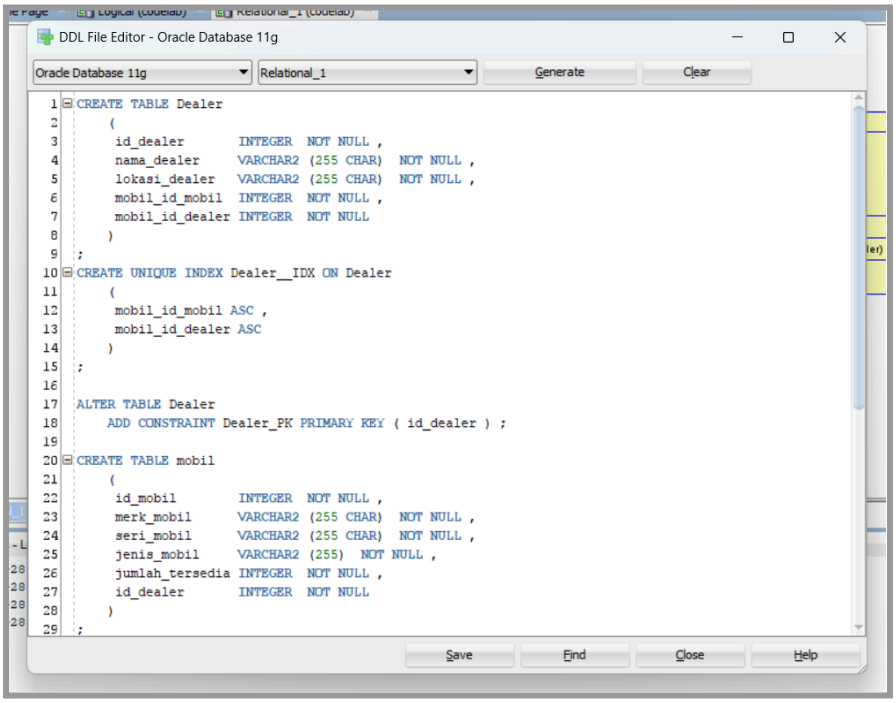
3. **Then, copy the entire SQL program.**



4. **Open the DDL File Editor and paste all the DDL you copied earlier.**



5. **Save the DDL file you have created in the location you want.**



6. **Upload the DDL file you created in SQL Developer to Oracle 11g xe and name the script you are uploading (To upload a DDL file to Oracle 11g xe, follow the same steps as in PRACTICE ACTIVITY instruction number 5).**
7. **Once you have successfully completed this, please proceed to Instructions 1 through 5. When you are done, show your results to the lab assistant. Keep up the good work!** 🤩

PRACTICUM GRADING RUBRIC

Assessment Aspects	Points
Query Dock	**Total 15%**
`INSERT INTO` correct	3%
Table name is correct	3%
Keyword VALUES correct	3%
All tables can be displayed	6%
Assignment	**Total 45%**
`INSERT` ≥ 3 data per table	15%
`SELECT` + `WHERE` correct	15%
`UPDATE` + `DELETE` correct	15%
Material Understanding	**Total 40%**
Explains query parts	10%
Explains table–column relation	10%
Explains `INSERT` logic	10%
Explains `UPDATE` / `DELETE` logic	10%

Important Notes on Practicum Implementation

During **Query Dock** week, students are encouraged to **ask questions** about module 4 material that they do not yet understand. This session not only tests syntax, but also helps students deepen their understanding of SQL concepts through discussions with assistants.

The **Query Dock score has a small percentage**, so the main focus of the assessment remains on **understanding the material** and the student's ability to explain the logic of the query. For each assessment item on the rubric:

- **100 → Correct and confident answer / clear explanation**
- **50 → Partially correct answer / still uncertain**
- **0 → Unable to answer / does not follow instructions**

With this scheme, assessment is carried out objectively and consistently according to the level of student mastery.